

2.1. Dada las siguientes transacciones escritas como Read/Write:

$T_1 = r_1(A); w_1(A); r_1(B); w_1(B)$

$T_2 = r_2(A); w_2(A); r_2(B); w_2(B)$

Nota: Si bien ambas parecen iguales en realidad pueden realizar operaciones diferentes sobre los ítems leídos.

(a) Incorporar locks para que llevarlas a un modelo con *locking* binario

(b) Realizar una historia legal pero no serializable

Historia legal

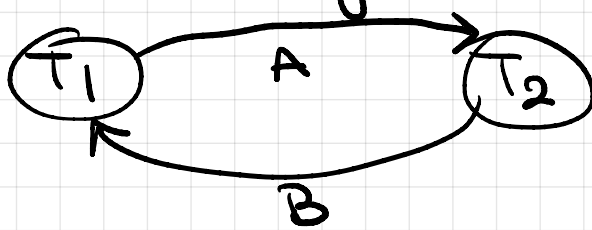
↳ Consistencia: $\Rightarrow T_i$ Puede leer o escribir un ítem x si previamente realizó un lock sobre x y no lo liberó.
 $\Rightarrow T_i$ realiza un lock sobre un elemento debe posteriormente liberarlo.

↳ Legalidad: T_i fue desea obtener un lock sobre x fue he sido lockeado por T_j en un modo que conflictiva, debe esperar hasta que T_j haga unlock de x .

a) $T_1 = l_1(A); r_1(A); w_1(A); u_1(A); l_1(B);$
 $r_1(B); w_1(B); u_1(B)$

$T_2 = l_2(A); r_2(A); w_2(A); u_2(A); l_2(B);$
 $r_2(B); w_2(B)$

b) Asumamos el caso de Precedencia



Quiero que
haya un ciclo
así lo que fue
no sea serializable.

T_1	T_2
$l_1(A)$ $r_1(A)$ $w_1(A)$ $u_1(A)$	
	$l_2(A)$ $r_2(A)$ $w_2(A)$ $u_2(A)$ $l_2(B)$ $r_2(B)$ $w_2(B)$ $u_2(B)$
$w_1(B)$ $r_1(B)$ $w_1(B)$ $u_1(B)$	